MICROBIT. УРОК №7. КРИЧИ ГРОМЧЕ!

Всем привет! Сегодня мы подключаем новый тип управления — микрофон вашего micro:bit V2 *(рис. 1)*. Плата не «понимает слова» и не распознаёт речь. Она измеряет уровень звука — насколько громко вокруг.

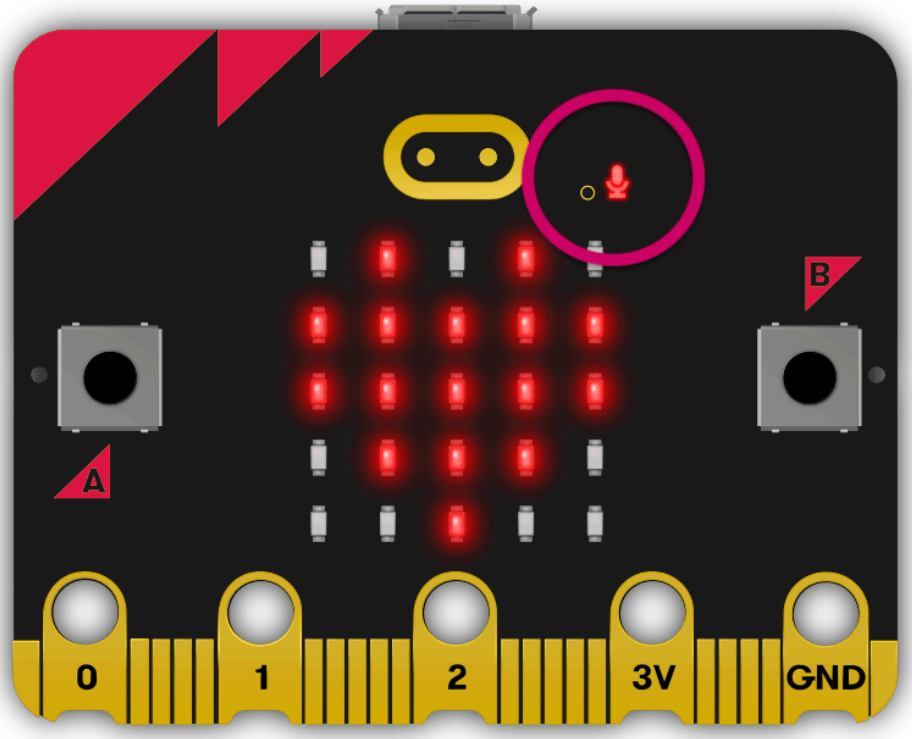


Рисунок 1

Интересный факт!

Слово «датчик» по-английски sensor. У датчиков почти всегда один вывод: они дают числовое значение. Даже кнопка — это датчик: она передает 0 (не нажата) или 1

> (нажата). Микрофон — тоже датчик, просто его числовое значение меняется плавнее.

> ***Историческая справка!*** *Громкость в акустике измеряют в децибелах (дБ). Но micro:bit V2 выдаёт относительный уровень 0…255. Это не дБ, но для игр отлично: мы сравниваем числа между собой и строим правила.*

Мини-словарик

- Уровень звука — число, которое становится больше, когда вокруг громче.
- Порог — граница: выше порога делаем действие, ниже — не делаем.
- Сглаживание — способ убрать дрожание и сделать управление мягким.
- Калибровка — короткая «настройка» под комнату: измеряем фон.
- Баланс — настройка чисел так, чтобы игра была и честной, и интересной.

Блиц-опрос

▸ **Что именно показывает число «скорость» в коде игры?**

▸ **В чем разница между скоростью и ускорением?**

▸ **Почему корабль не может остановиться мгновенно, как только мы нажали на тормоз**

▸ **Зачем в игре на экране 5×5 использовать систему «фикс-поинт» (когда 1 клетка = 10 единиц)?**

▸ **Какие три физические величины необходимы для создания правдоподобного движения?**

Микрофон реагирует на колебания воздуха. Когда вы говорите, шепчете, хлопаете или двигаете стул, воздух колеблется *(рис. 2)*. Микрофон превращает колебания в электрический сигнал. micro:bit переводит сигнал в число — его мы и используем.

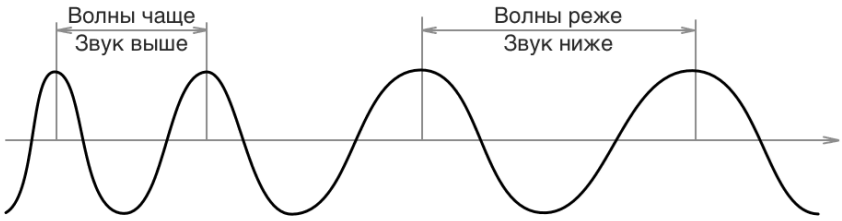


Рисунок 2

Главное! Число не обязано быть идеально точным, чтобы давать результат. Нам достаточно, что оно корректно меняется в ту или иную сторону: стало громче — число выросло; стало тише — число уменьшилось. Это называется **относительное измерение**.

Начнём с индикатора. Он рисует столбик на экране: чем громче звук, тем выше столбик. Эта мини-программа помогает подобрать пороговые значения под вашу окружающую обстановку.

```
from microbit import *

smooth = 0

while True:
    level = microphone.sound_level()      # 0..255
    smooth = (smooth * 3 + level) // 4   # сглаживание

    bars = (smooth * 5) // 255           # 0..5 уровней

    display.clear()
    for y in range(5):
        if y >= 5 - bars:
            display.set_pixel(2, y, 9)
```

```
        sleep(50)
```

level — «сырая громкость», она может прыгать. **smooth** — «плавная громкость». Мы делаем **smooth** из старого значения и нового уровня, чтобы убрать дрожание. Дальше мы превращаем 0…255 в 0…5 уровней и рисуем столбик на колонке x=2.

Обратите внимание: мы используем целочисленное деление //, потому что экран работает с целыми уровнями. Например, bars может быть только 0,1,2,3,4,5 — половинок столбика на 5×5 нет.

На micro:bit нам часто нужно превращать большое число в маленькое. Для этого удобно использовать **целочисленное деление //**: оно отбрасывает дробную часть. А знак **% (остаток от деления)** помогает делать повторяющиеся ритмы: например, ворота открываются и закрываются «по кругу».

Например, если **period = 800**, то (**time % 800**) всегда будет в диапазоне 0…799. Получается «секундомер», который постоянно сбрасывается. Мы можем сказать: первые 400 мс ворота открыты, следующие 400 мс закрыты — и так бесконечно.

Почему микрофон «шумит»? Потому что вокруг почти никогда не бывает идеальной тишины. Есть кулер компьютера, разговоры в коридоре, шорох одежды, движение рук. Если мы будем превращать каждый маленький шум в движение, игра станет неуправляемой.

Сглаживание помогает: значение меняется мягко, и случайные пики становятся менее опасными. Но сглаживание не отменяет пороги. Поэтому мы используем и фильтр, и порог **MOVE_DELTA**.

В основной игре игрок едет по дорожке из 5 клеток. На пути стоят 3 ворот. Ворота могут быть открыты или закрыты. Если вы въехали в закрытые — проиграли. Если дошли до финиша (последняя клетка) — победили.

Голос управляет скоростью. Это значит, что у вас появляется «газ» и «тормоз» одновременно: громче — быстрее, тише — медленнее. Чтобы пройти ворота, часто выгодно остановиться на клетке перед воротами и дождаться открытия. Это тренирует терпение и стратегию.

Шаг 1. Импорты и параметры

Сначала подключаем micro:bit и gamepad. Вводим параметры. MOVE_DELTA — насколько мы должны быть громче фона, чтобы поехать. MAX_SPEED10 — ограничение скорости (в десятых долях клетки).

```
from microbit import *
import gamepad

TRACK_Y = 2
FRAME_MS = 70

MOVE_DELTA = 18
MAX_SPEED10 = 6
```

Почему скорость в «десятых»? Потому что так проще сделать плавность. Если хранить положение только как 0,1,2,3,4, мы будем прыгать по клеткам без промежутков. А если 1 клетка = 10 единиц, то можно копить движение постепенно: 0→1→2… и только иногда переходить на новую клетку.

Как выбрать **MOVE_DELTA**? Если игра едет от любого шороха — MOVE_DELTA слишком маленький. Если игра не едет даже когда вы говорите — MOVE_DELTA слишком большой или калибровка поймала шум.

Шаг 2. Ворота и их ритм

Опишем ворота. Каждый объект хранит: на какой клетке он стоит (x), период переключения и фазу (сдвиг). Фаза нужна, чтобы ворота не работали одинаково — так играть интереснее.

```
GATES = [
    (1, 700, 0),
    (2, 900, 200),
    (3, 800, 400),
]

def gate_is_open(x, t_ms):
    for gx, period, phase in GATES:
        if gx == x:
            m = (t_ms + phase) % period
            return m < (period // 2)
    return True
```

Сначала мы ищем нужные ворота по x. Затем считаем m — «секунда внутри периода». Если m меньше половины периода — ворота открыты. Иначе закрыты. Так мы получаем простые «мигающие» ворота без сложной математики.

Шаг 3. Сглаживание и скорость из громкости

Теперь самое важное: превращаем микрофон в скорость. Сначала сглаживаем уровень, затем

вычитаем шумовой фон и порог. Если получился маленький diff — стоим. Если большой — едем.

```
smooth = 0
noise_floor = 0

def read_sound_smooth():
    global smooth
    level = microphone.sound_level()
    smooth = (smooth * 3 + level) // 4
    return smooth

def speed10_from_sound(level):
    diff = level - (noise_floor + MOVE_DELTA)
    if diff <= 0:
        return 0
    if diff > 200:
        diff = 200
    return (diff * MAX_SPEED10) // 200
```

diff — «насколько мы громче тишины». Если diff <= 0, значит тише порога — скорость 0. Если diff растёт, скорость растёт. Мы делим на 200, чтобы превратить большой диапазон diff в маленький диапазон скорости.

Почему именно 200? Это удобное число для масштаба. Можно выбрать 150 или 220 — главное, чтобы управление было приятным. Так в разработке игр часто делают: сначала выбирают «примерные» числа, потом тестируют и подстраивают.

Шаг 4. Калибровка тишины

Калибровка — это короткая настройка под помещение. Мы измеряем средний шум 1–2 секунды и считаем noise_floor. Если класс шумный, noise_floor

будет выше. Это нормально — просто пороги будут считаться от этого фона.

```
def calibrate_noise():
total = 0
samples = 0
start = running_time()
while running_time() - start < 1200:
    total += microphone.sound_level()
    samples += 1
    sleep(30)
if samples > 0:
    return total // samples
return 0
```

Мы берём много измерений и усредняем. Это как «средний шум за секунду». Если в момент калибровки кто-то хлопнул, среднее станет выше — поэтому важно договориться и 1 секунду помолчать.

Шаг 5. Рисуем мир игры

Экран 5×5 маленький, поэтому мы рисуем очень просто: ворота, игрок и финиш. Чтобы игрок понимал скорость, добавим индикатор яркости в левом верхнем углу.

```
def draw_world(player_x, t_ms, speed10):
display.clear()
display.set_pixel(0, 0, min(9, speed10 + 2))

for gx, _, _ in GATES:
    if gate_is_open(gx, t_ms):
        display.set_pixel(gx, TRACK_Y, 2)
    else:
        display.set_pixel(gx, TRACK_Y, 9)
```

```
display.set_pixel(4, TRACK_Y, 5)
display.set_pixel(player_x, TRACK_Y, 9)
```

Закрытые ворота яркие (9), открытые тусклые (2). Финиш — средней яркости (5). Игрок всегда яркий (9). Индикатор скорости показывает: почему вы вдруг начали быстро ехать или наоборот остановились.

Шаг 6. Победа и поражение

```
def crash():
    gamepad.vibrate(220)
    display.show(Image.SAD)
    sleep(900)

def win():
    display.show(Image.HAPPY)
    sleep(900)
```

При проигрыше используем вибрацию геймпада. Музыку не включаем: у вибромотора pin0, и модуль music часто тоже использует pin0, из-за чего мотор может включаться вместе с мелодией.

Финальный main.py целиком

Собираем всё вместе.

```
from microbit import *
import gamepad

TRACK_Y = 2
FRAME_MS = 70
MOVE_DELTA = 18
```

```
MAX_SPEED10 = 6

GATES = [(1, 700, 0), (2, 900, 200), (3, 800, 400)]
smooth = 0
noise_floor = 0

def gate_is_open(x, t_ms):
    for gx, period, phase in GATES:
        if gx == x:
            m = (t_ms + phase) % period
            return m < (period // 2)
    return True

def read_sound_smooth():
    global smooth
    level = microphone.sound_level()
    smooth = (smooth * 3 + level) // 4
    return smooth

def speed10_from_sound(level):
    diff = level - (noise_floor + MOVE_DELTA)
    if diff <= 0:
        return 0
    if diff > 200:
        diff = 200
    return (diff * MAX_SPEED10) // 200

def calibrate_noise():
    total = 0
    samples = 0
    start = running_time()
    while running_time() - start < 1200:
        total += microphone.sound_level()
        samples += 1
        sleep(30)
    if samples > 0:
        return total // samples
    return 0

def draw_world(player_x, t_ms, speed10):
    display.clear()
    display.set_pixel(0, 0, min(9, speed10 + 2))
    for gx, _, _ in GATES:
        display.set_pixel(gx, TRACK_Y, 2 if gate_is_open(
    display.set_pixel(4, TRACK_Y, 5)
    display.set_pixel(player_x, TRACK_Y, 9)
```

```
def crash():
    gamepad.vibrate(220)
    display.show(Image.SAD)
    sleep(900)

def win():
    display.show(Image.HAPPY)
    sleep(900)

display.scroll("CAL")
noise_floor = calibrate_noise()

pos10 = 0
player_x = 0
last_frame = running_time()

while True:
    now = running_time()

    if now - last_frame >= FRAME_MS:
        last_frame = now

        level = read_sound_smooth()
        speed10 = speed10_from_sound(level)

        pos10 += speed10
        new_x = pos10 // 10

        while player_x < new_x and player_x < 4:
            next_x = player_x + 1

            if not gate_is_open(next_x, now):
                crash()
                pos10 = 0
                player_x = 0
                break

            player_x = next_x

            if player_x == 4:
                win()
                pos10 = 0
                player_x = 0
                break
```

```
        draw_world(player_x, now, speed10)

    sleep(10)
```

После калибровки (CAL) игра идёт кадрами. Каждый кадр мы считаем скорость и добавляем её в pos10. Когда pos10 становится достаточно большим, игрок переходит на следующую клетку. Почему проверяем переходы в цикле while? Потому что за один кадр игрок может попытаться пройти сразу несколько клеток. Мы проверяем по очереди, чтобы ворота нельзя было «перепрыгнуть». Если ворото закрыто — crash(): вибрация и грустный смайл, затем сброс. Если дошли до x=4 — win(): улыбка и новый раунд.

